

Rabin-Karp Algorithm

In the Rabin-Karp method, We'll produce a hash of the pattern we're looking for and see if the rolling hash of our text fits it or not. We can be certain that the pattern does not exist in the text if it does not match. The pattern can, however, be found in the text if it matches. Consider the following scenario:

Let's assume we have the text **yeminsajid** and want to see if the pattern **nsa** appears anywhere in it. We'll need a prime number to calculate the hash and rolling hash. Any prime number will work. For this example, let's choose prime = 11. This formula will be used to calculate the hash value:

$$(1\text{st letter}) \times (\text{prime}) + (2\text{nd letter}) \times (\text{prime})^1 + (3\text{rd letter}) \times (\text{prime})^2 \\ \times + \dots$$

Let's define lookup table:

a -> 1	g -> 7	m -> 13	s -> 19	y -> 25
b -> 2	h -> 8	n -> 14	t -> 20	z -> 26
c -> 3	i -> 9	o -> 15	u -> 21	
d -> 4	j -> 10	p -> 16	v -> 22	
e -> 5	k -> 11	q -> 17	w -> 23	
f -> 6	l -> 12	r -> 18	x -> 24	

The hash value of **nsa** will be:

$$14 \times 11^0 + 19 \times 11^1 + 1 \times 11^2 = 344$$

Now, let's calculate the text's rolling-hash.

We need to verify whether the strings match or not if the rolling hash matches the hash value of our pattern.

yeminsajid - text

Because our pattern consists of three characters, we'll compute the hash value using the first three letters of our text, **yem**. We obtain the following:

$$25 \times 11^0 + 5 \times 11^1 + 13 \times 11^2 = 1653$$

This result does not match the hash value of our pattern. As a result, the string isn't present.

Now it's time to think about the next move.

To find the hash value of **emi**, our next string. Using our formula, we can figure this out. But that would be a waste of time and resource. Instead, Let us choose a different strategy.

We remove the value of the previous string's first letter from our current hash value. In this situation, the answer is y. We get, (yem)

$$1653 - 25 = 1628$$

- We divide the difference with our **prime**, which is **11** for this example. We get, $1628 / 11 = 148$.
- We add **new letter X (prime)^{m-1}**, where **m** is the length of the pattern, with the quotient, which is **i = 9**. We get, $148 + 9 \times 11^2 = 1237$.
- **yeminsajid - text**

The new hash value is not equal to our patterns hash value. Moving on, for **n** we get:

Previous String: emi

First Letter of Previous String: e(5)

New Letter: n(14)

New String: "min"

$$1237 - 5 = 1232$$

$$1232 / 11 = 112$$

$$112 + 14 \times 11^2 = 1806$$

It doesn't match. After that, for **s**, we get:

Previous String: **min**

First Letter of Previous String: m(13)

New Letter: s(19)

New String: "ins"

$$1806 - 13 = 1793$$

$$1793 / 11 = 163$$

$$163 + 19 \times 11^2 = 2462$$

It doesn't match. Next, for **a**, we get:

Previous String: **ins**

First Letter of Previous String: i(9)

New Letter: a(1)

New String: "nsa"

$$2462 - 9 = 2453$$

$$2453 / 11 = 223$$

$$223 + 1 \times 11^2 = 344$$

There is a match.

The pseudo-code will look like this:

Hash Calculation:

```
Procedure Calculate-Hash(String, Prime, x):  
hash := 0 // Here x denotes the length to be  
considered  
for m from 1 to x // to find the hash value  
    hash := hash + (Value of String[m])m-1  
end for  
Return hash
```

Hash Recalculation:

```
Procedure Recalculate-Hash(String, Curr, Prime, Hash):  
Hash := Hash - Value of String[Curr] //here Curr denotes First Letter of  
Previous String  
Hash := Hash / Prime  
m := String.length  
New := Curr + m - 1  
Hash := Hash + (Value of String[New])m-1  
Return Hash
```

String Match:

```
Procedure String-Match(Text, Pattern, m):  
for i from m to Pattern-length + m - 1  
    if Text[i] is not equal to Pattern[i]  
        Return false  
    end if  
end for  
Return true
```

Rabin-Karp:

```
Procedure Rabin-Karp(Text, Pattern, Prime):  
  m := Pattern.Length  
  HashValue := Calculate-Hash(Pattern, Prime, m)  
  CurrValue := Calculate-Hash(Pattern, Prime, m)  
  for i from 1 to Text.length - m  
    if HashValue == CurrValue and String-Match(Text, Pattern, i) is true  
      Return i  
    end if  
    CurrValue := Recalculate-Hash(String, i+1, Prime, CurrValue)  
  end for  
  Return -1
```